

CSY3056 AS1 Scaffold Guidance

Using the Starter Files Effectively

Purpose

The AS1 scaffold is provided as a starting point so that everyone has a basic technical structure to work from.

It is not a complete solution, and it is not intended to restrict your design. You may adapt or reorganise the structure where appropriate, as long as your final submission clearly demonstrates the required Python implementation, automated testing, Jenkins-based CI/CD evidence, Docker evidence, and explanation of your DevOps decisions.

The placeholder files are only there to help you begin. They are not sufficient for a strong submission.

Using the Scaffold

The scaffold gives you a basic starting structure for your work. You should adapt it to fit your own chosen software system and component.

Your final project should show that you can:

- Implement a meaningful Python component
- Write relevant automated tests
- Use Jenkins to support CI/CD evidence
- Use Docker to support reproducible execution
- Explain how your technical choices support reliable software delivery

The assessment is not about building a large application. It is about showing how a focused software component can be developed, tested, automated, and explained professionally.

Choosing a Meaningful Component

Your project should begin with a larger software system idea, but you are only implementing one focused component from that system.

A strong component should normally involve some form of:

- Decision-making
- Validation
- Calculation
- Classification
- Checking rules

- Handling different outcomes

Avoid choosing a component that only displays text or returns a fixed value.

For example, instead of building a full academic misconduct management system, you might implement a case-priority component that classifies a report as low, medium, or high priority based on factors such as evidence strength, repeat offence indicator, submission type, and deadline sensitivity.

This is an example only. You should choose a component that fits your own proposed system and can be tested meaningfully.

Testing Expectations

Your tests should show that your component works under different conditions.

Do not only test the easiest case. You should consider:

- Normal expected inputs
- Boundary or edge cases
- Invalid inputs
- Failure or risk cases
- Different decision paths in your logic

There is no fixed number of tests. The important issue is whether the tests are relevant and whether they demonstrate confidence in the component. The assessment rewards the relevance of tests, coverage of different logic paths, and evidence that failures are detected through CI/CD.

Jenkins and Docker Expectations

The scaffold provides a basic starting point, but you are expected to adapt it.

Your Jenkins evidence should show that the pipeline can:

- Run automatically
- Execute relevant tests
- Fail when there is a problem
- Pass when the problem is fixed

Your Docker evidence should show that your project can be built or executed in a reproducible way.

You should be able to explain what your Jenkinsfile and Dockerfile do. Do not submit them as unexplained technical artefacts.

Use of Different Environments

You are not compulsorily required to fully implement Dev, Test, Staging, and Production environments for AS1.

However, stronger submissions may explain how the component would move through different environments in a real DevOps workflow. Where appropriate, you may demonstrate simple environment-aware behaviour using Jenkins stages, Docker configuration, environment variables, or additional tools such as Terraform.

Evidence to Collect

Collect evidence while you work, not only at the end.

Useful evidence may include:

- Tests passing locally
- Tests failing when you introduce an error
- Jenkins failed run
- Jenkins successful run
- Docker build result
- Docker run or execution result
- Screenshots of key logs or outputs
- Short explanation of what each piece of evidence proves

Your submitted evidence should help show that your automation is working and that your component is being checked properly.

Going Beyond the Minimum

Stronger submissions may show:

- A clear link between the proposed system and the implemented component
- Better coverage of normal, edge, and failure cases
- Meaningful CI/CD failure and recovery evidence
- Clear explanation of Docker and reproducibility
- A simple DevOps or system architecture diagram
- Thoughtful discussion of environment separation, release risk, or infrastructure
- Optional discussion of how GitHub, Terraform, or other tools could support the system in a real deployment

Additional tools are allowed where appropriate, but they should support your explanation rather than replace the required tools.